FULL TECHNICAL DOCUMENT

Central Java Youth Sustainability Competition 2026

Innovation of IoT and Solar Panel-Based Shrimp Pond Control System

in the Coastal Area of Kebumen, Central Java

Research & Development Team

Sufi Anugrah

Abu Yazid Bustomi

Project Repository:

`github.com/sufiarh/sistem-smart-tambak`

Tambakmulyo, Kebumen, Central Java, Indonesia | 2026

CHAPTER 1 — SYSTEM IDENTITY & OVERVIEW

1.1 System Identity

System Title	:	Innovation of IoT and Solar Panel-Based Shrimp Pond Control System in the Coastal Area of Kebumen, Central Java
Study Case Location	:	Tambakmulyo, Kebumen, Central Java, Indonesia
Commodity	:	Pacific White Shrimp (Litopenaeus vannamei)
Pond Size	:	50m × 40m = 2,000 m²
Water Depth	:	1.2 – 1.5 meters
Capacity	:	± 100,000 shrimp
Energy Source	:	Solar Power System (SPS) 6 kWp — Fully Off-Grid (no PLN dependency)
System Brain	:	STB HG680P (Armbian Linux) — Go + SQLite + Cloudflare Tunnel
Programming Language	:	Go (Golang) — compiled binary, 3 parallel goroutines
Remote Access	:	Cloudflare Tunnel → farmer's domain (HTTPS encrypted)
Repository	:	github.com/sufiarh/sistem-smart-tambak
R&D Team	:	Sufi Anugrah | Abu Yazid Bustomi
Competition	:	Central Java Youth Sustainability Competition 2026

1.2 Background

Kebumen Regency, Central Java, has a coastline directly bordering the Indian Ocean and is one of Indonesia's largest Pacific white shrimp farming centers. The Tambakmulyo area was officially inaugurated by the President of the Republic of Indonesia as a location for the Kawasan-Based Shrimp Aquaculture (BUBK) program, serving as a national model for modern and sustainable pond management.

However, most farmers in Kebumen's coastal areas still manage their ponds using conventional and manual methods. Water quality monitoring is conducted only 1–2 times per day using simple tools. As a result, sudden changes in parameters —

such as dissolved oxygen drops in the early morning hours — often go undetected until it is too late, causing mass shrimp mortality and significant economic losses.

Furthermore, the remote coastal location of these ponds means that access to PLN (state electricity) is often unstable or entirely unavailable. This drives the need for an independent, sustainable, and environmentally friendly energy source — directly aligned with the theme of the Central Java Youth Sustainability Competition 2026 on energy transition.

This system presents an integrated solution: combining Internet of Things (IoT) technology with Solar Power Systems (SPS) to create a smart, efficient, and sustainable shrimp pond that operates entirely off-grid.

1.3 System Architecture — 3 Main Layers

Layer 1 — Energy: Solar Panels → MPPT SCC → LiFePO4 Battery → Pure Sine Wave Inverter → AC 220V to all systems.

Layer 2 — Control (Control House): STB HG680P runs a Go binary with 3 parallel goroutines. USB Port 1 → USB Relay Module (actuator control). USB Port 2 → USB to RS485 Converter (sensor reading).

Layer 3 — Field (Pond): Seven RS485 sensors in daisy-chain configuration inside a 50m × 40m pond. Six actuators operate automatically based on real-time sensor trigger logic.

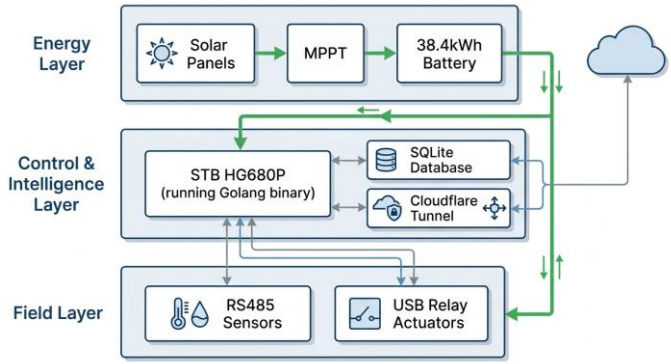


Figure 1. System Block Diagram — 3 Integrated Layers

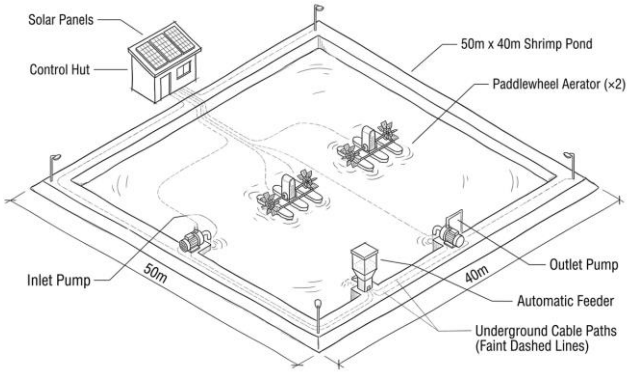


Figure 2. 3D Isometric System Overview — Macro View of the Entire System

CHAPTER 2 — LAYER 1: SOLAR POWER SYSTEM (SPS)

2.1 Power Consumption Calculation

Before designing the SPS, the total power requirements of all pond equipment were calculated in detail. The paddle wheel aerators constitute the largest load because they must operate intensively to maintain optimal dissolved oxygen levels for the shrimp.

Device	Qty	Power	Active Hours	kWh/Day	Notes
Paddle Wheel Aerator	2	750 W	12 hrs	18 kWh	Largest load in system
Inlet Pump	1	370 W	2 hrs	0.74 kWh	Triggered by water quality
Outlet Pump	1	370 W	1 hr	0.37 kWh	Triggered by turbidity
Auto Feeder	1	50 W	0.5 hr	0.025 kWh	Schedule + DO check
Lighting (4 points)	4	20 W	12 hrs	0.96 kWh	Triggered by light sensor
HG680P + Router	1	30 W	24 hrs	0.72 kWh	Always on — never cut
TOTAL				**≈ 20.8 kWh**	Daily system requirement

SPS Capacity Calculation: 20.8 kWh/day ÷ 4.5 hours peak sun (Kebumen coast) ÷ 80% system efficiency = 5,778 Wp → rounded up to 14 panels × 430 Wp = 6,020 Wp ≈ 6 kWp.

2.2 SPS Components

Component	Specification	Main Function
Monocrystalline Solar Panel	14 panels × 430 Wp, 48V	Converts sunlight → DC electricity 48V
MPPT Solar Charge Controller	2 units × 60A, 48V	Regulates battery charging, prevents overcharge/over-discharge

LiFePO4 Battery	4 units × 200Ah 48V = 38.4 kWh	Stores energy for nighttime & cloudy days (2-night backup)
Pure Sine Wave Inverter	5,000W, 48V DC → 220V AC	Converts DC → clean 220V AC, safe for all equipment
Smart MCB Panel	7 channel controlled MCBs	Distributes & controls power to each actuator independently

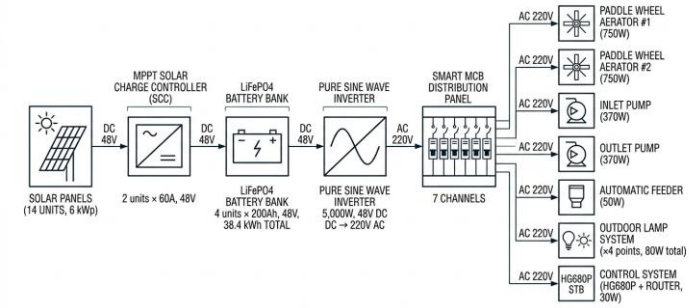


Figure 3. SPS Wiring Schematic — Solar Panel → SCC → Battery → Inverter → MCB Panel

2.2.1 Monocrystalline Solar Panels

Monocrystalline panels are chosen for their highest energy conversion efficiency (18–22%) compared to polycrystalline or thin-film alternatives. With 14 panels at 430 Wp each, total generation capacity reaches 6,020 Wp. Panels are mounted on the control house rooftop and supplementary poles at an optimal north-facing tilt angle to maximize solar exposure throughout the day in Kebumen's coastal region.

2.2.2 MPPT Solar Charge Controller (SCC)

The MPPT (Maximum Power Point Tracking) Solar Charge Controller dynamically tracks the panel's maximum power point, improving charging efficiency by 20–30% over PWM-type controllers. Two units of 60A SCC protect the batteries from overcharging and over-discharging, which would permanently damage the battery cells.

2.2.3 LiFePO4 Battery Bank

Four LiFePO4 (Lithium Iron Phosphate) battery units at 200Ah 48V each provide a total capacity of 38.4 kWh — sufficient to power the entire system for over 2

nights without sunlight. LiFePO4 is preferred over conventional lead-acid batteries due to its 10-year lifespan, tolerance for high coastal temperatures, deep discharge resilience, and fire safety.

2.2.4 Pure Sine Wave Inverter

A 5,000W Pure Sine Wave inverter converts DC 48V to clean AC 220V identical to grid electricity. This type is essential — Modified Sine Wave inverters can damage aerator motors and pumps over time due to harmonic distortion in the waveform.

2.2.5 Smart MCB Panel — Channel Mapping

- MCB 1 → Paddle Wheel Aerator #1 (750 Watts)
- MCB 2 → Paddle Wheel Aerator #2 (750 Watts)
- MCB 3 → Inlet Water Pump (370 Watts)
- MCB 4 → Outlet Water Pump (370 Watts)
- MCB 5 → Auto Feeder (50 Watts)
- MCB 6 → Lighting System — 4 points (80 Watts total)
- MCB 7 → HG680P + WiFi Router (30 Watts) — never disconnected

CHAPTER 3 — LAYER 2: CONTROL HOUSE (SYSTEM BRAIN)

All control equipment is housed in a centralized control house built at the edge of the pond. This structure contains the SPS components, power distribution panel, and computing and communication devices.

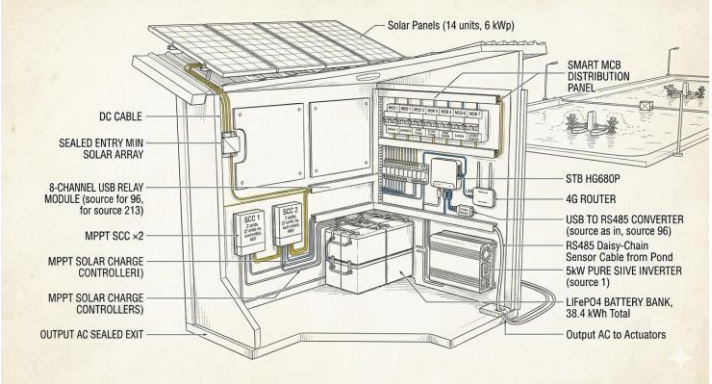


Figure 4. Control House Interior Layout — All Components in One Space

3.1 STB HG680P as the Main Processing Unit

The primary innovation of this system is the repurposing of a Set Top Box (STB) HG680P as a Single Board Computer (SBC). The HG680P is flashed with Armbian Linux — an open-source Debian-based operating system for ARM devices — transforming it into a fully functional Linux mini-computer at a fraction of the cost of a Raspberry Pi.

Device	Set Top Box (STB) HG680P
Operating System	Armbian Linux (replacing factory OS)
Processor	ARM Cortex-A53 Quad-core 1.5 GHz
RAM	2 GB
Storage	MicroSD 32 GB
Connectivity	Built-in WiFi, 2× USB ports
Price	± IDR 200,000 – 300,000 (~USD 12–18)
Programming Language	Go (Golang) — 1 binary, 3 parallel goroutines

Database	SQLite — historical sensor data storage
Remote Access	Cloudflare Tunnel → HTTPS farmer's domain
USB Port 1	USB Relay Module 8 Channel (actuator control via HID protocol)
USB Port 2	USB to RS485 Converter CH340/FT232 (sensor reading via Modbus RTU)
Innovation Value	Cheap STB converted to Linux SBC — replicable by small-scale farmers

Innovation Value: The HG680P, available for USD 12–18, proves that a smart pond system can be implemented affordably and replicated by small-scale farmers without relying on expensive imported hardware.

3.2 Software Architecture — Go (Golang)

The entire system logic is written in Go (Golang) and compiled into a single binary that runs on Armbian Linux. Go is chosen because it is a compiled language that produces extremely lightweight and efficient programs — ideal for resource-constrained hardware like the HG680P. Three goroutines run in parallel using Go's built-in concurrency, ensuring sensor reading, control decision-making, and dashboard serving occur simultaneously without conflict.

Goroutine	Name	Responsibility
Goroutine 1	**Sensor Reader**	Reads 7 RS485 sensors via Modbus RTU every 5 seconds, saves to SQLite
Goroutine 2	**Controller**	Reads SQLite, compares thresholds, sends commands to USB Relay Module
Goroutine 3	**Web Server**	HTTP server port 8080, REST API for sensor & relay data, serves dashboard

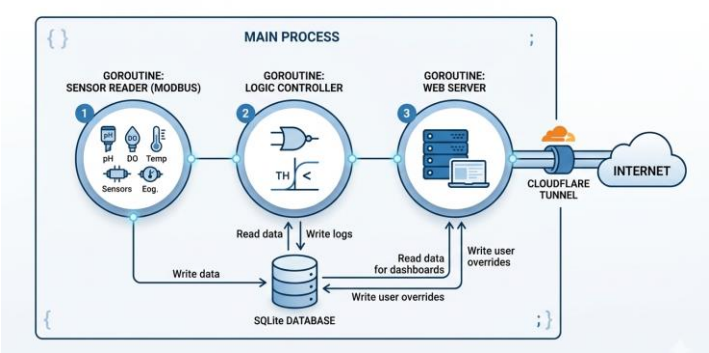


Figure 5. Go Software Architecture Diagram — 3 Parallel Goroutines + SQLite + Cloudflare

3.2.1 Goroutine 1 — Sensor Reader

Runs in an endless loop, reading all 7 RS485 sensors via Modbus RTU every 5 seconds and saving data to SQLite. The 5-second interval balances real-time monitoring accuracy with computational efficiency.

3.2.2 Goroutine 2 — Controller

The intelligence core of the system. Every cycle, it reads the latest sensor values from SQLite, compares them against optimal thresholds for Litopenaeus vannamei, and sends commands to the USB Relay Module to activate or deactivate the appropriate actuator. Logic is purely threshold-based trigger — no fixed schedules except for the auto feeder which follows the shrimp's natural feeding rhythm.

3.2.3 Goroutine 3 — Web Server

Runs an HTTP server on port 8080 serving the real-time dashboard, sensor data REST API, and manual relay control interface. Accessible from the farmer's smartphone from anywhere in the world via Cloudflare Tunnel.

3.2.4 SQLite Database

SQLite operates as a single file on the MicroSD card without requiring a separate database server. Sensor data from all 7 sensors is stored every 5 seconds with timestamps. The dashboard uses per-hour SQL aggregation for efficient 24-hour historical charts without overwhelming the display with thousands of data points.

3.3 USB Connections — 2 Ports Used to Full Capacity

The HG680P has only 2 USB ports, but both are used to maximum effect: sensors are daisy-chained (all 7 sensors on 1 cable) and actuators are controlled via USB HID protocol — no GPIO pins required.

3.3.1 USB Port 1 — USB Relay Module 8 Channel

Controls 6 Smart MCBs (aerators, pumps, feeder, lighting) via HID protocol. Go sends channel-specific ON/OFF commands → the relay module closes or opens the 220V AC circuit for the corresponding actuator. Two spare channels are available for future expansion.

3.3.2 USB Port 2 — USB to RS485 Converter

The CH340 or FT232RL chip converts USB Serial signals into RS485 differential signals (2-wire: A+/B-). Modbus RTU protocol allows all 7 sensors to share a single cable, each identified by a unique address (01–07). Go polls each sensor sequentially every 5 seconds.

3.4 Cloudflare Tunnel — Secure Remote Access

The cloudflared daemon runs as a background service on the HG680P, connecting the local web server (port 8080) to the farmer's domain via HTTPS encryption. Farmers can access the dashboard from any smartphone without port forwarding, static IP, or VPN configuration. All traffic is automatically encrypted by Cloudflare's infrastructure.

CHAPTER 4 — LAYER 3: SENSORS & ACTUATORS IN THE POND

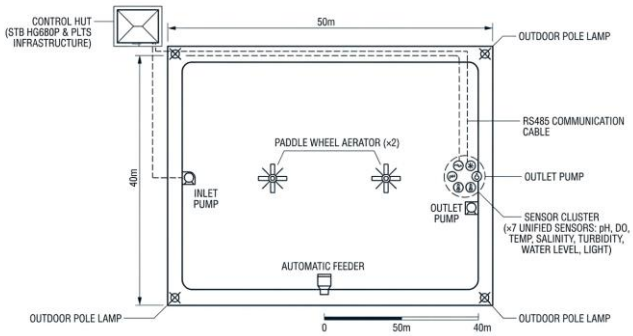


Figure 6. Pond Top-View Layout — Positions of Sensors, Actuators, and Control House

4.1 Study Case Pond Specification

The study case involves a single pond measuring 50m × 40m = 2,000 m², water depth 1.2–1.5 meters, with a capacity of approximately 100,000 Pacific white shrimp. This size is representative of small-to-medium scale ponds commonly found in Tambakmulyo, Kebumen. The entire sensor and actuator system is designed specifically for this single pond unit.

4.2 RS485 Sensor System — Daisy-Chain Configuration

All sensors used are RS485 Modbus type — industrial-grade sensors that output digital data directly without requiring analog-to-digital conversion. Sensors are connected in a daisy-chain configuration: all 7 sensors share a single RS485 cable with unique Modbus addresses (01–07), allowing the HG680P to read all sensors through just one USB to RS485 Converter.

Sensor	RS485 Address	Measures	Normal Threshold	Risk If Deviated
pH Sensor	01	Water acidity level	7.5 – 8.5	Gill damage, stress

DO Sensor	02	Dissolved oxygen	≥ 4.0 mg/L	Mass death within hours
Temperature Sensor	03	Water temperature	26 – 30°C	Stunted growth
Salinity/EC Sensor	04	Salt concentration	15 – 25 ppt	Osmotic stress
Turbidity Sensor	05	Water clarity	< 50 NTU	Bacterial outbreak indicator
Water Level Sensor	06	Pond water height	≥ 1.0 meter	Leakage, drought risk
Light Sensor	07	Ambient light intensity	Day / Night	Triggers auto lighting

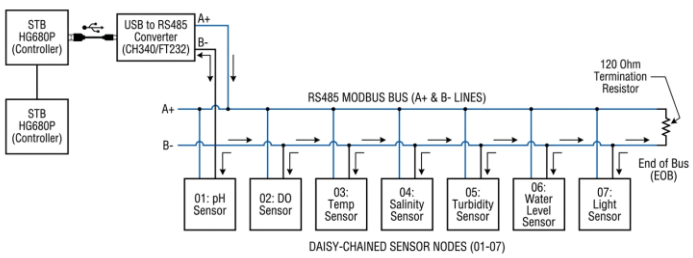


Figure 7. RS485 Daisy-Chain Wiring Schematic — 7 Sensors on a Single Cable

RS485 Advantages over analog sensors: Direct digital output (no ADC needed), high noise immunity over long cable runs, all sensors on one cable, and no GPIO pins required — works perfectly with just a USB port.

4.3 Actuator System — Trigger-Based Control

Six actuators are automatically controlled via Smart MCBs using the USB Relay Module. Each actuator has its own dedicated trigger logic based on the relevant sensor parameter.

Actuator	Relay Ch.	MCB	Power	Control Trigger
Paddle Wheel Aerator #1	CH1	MCB 1	750 W	DO < 4.0 mg/L → ON | DO > 5.5 mg/L → OFF

Paddle Wheel Aerator #2	CH2	MCB 2	750 W	Same as Aerator #1 (activated simultaneously)
Inlet Water Pump	CH3	MCB 3	370 W	pH < 7.5 or > 8.5 | Salinity > 25 ppt | Level < 1m → ON
Outlet Water Pump	CH4	MCB 4	370 W	Turbidity > 50 NTU → ON | Turbidity < 20 NTU → OFF
Auto Feeder	CH5	MCB 5	50 W	Schedule 07:00/12:00/17:00 + DO ≥ 4.0 mg/L → ON 30 sec
Lighting System	CH6	MCB 6	80 W	Light sensor dark → ON | Light sensor bright → OFF

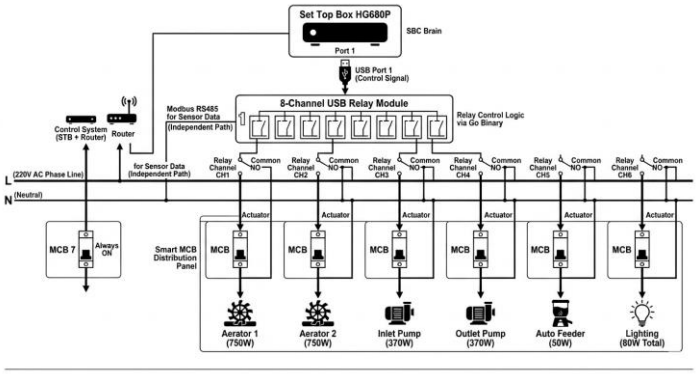


Figure 8. Full Control System Schematic — HG680P → USB Relay → MCB → Actuators

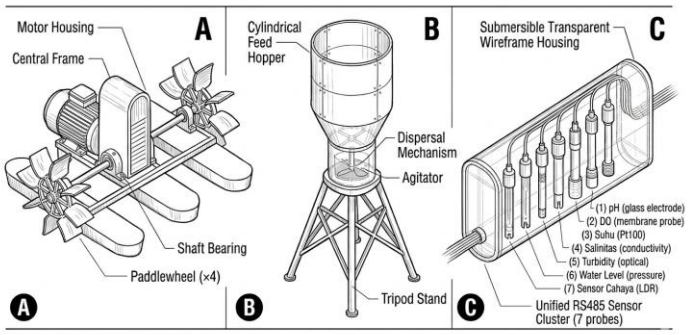


Figure 9. Actuator & Sensor Hardware Gallery — Physical System Components

4.3.1 Paddle Wheel Aerators

Two 750W paddle wheel aerators create water splashing that absorbs atmospheric oxygen into the pond water. Aerators are the most critical actuator (DO below threshold = mass mortality within hours) and the largest power consumer (60% of

total consumption). Automatic DO-based control is crucial for PLTS energy efficiency.

4.3.2 Inlet & Outlet Water Pumps

The inlet pump introduces fresh water when pH, salinity, or water level deviates from thresholds. The outlet pump discharges dirty water when turbidity is too high (indicating excess bacterial load or metabolic waste). Both operate independently based on different sensor parameters.

4.3.3 Auto Feeder

The feeder activates on schedule (07:00, 12:00, 17:00) with one mandatory condition: DO must be ≥ 4.0 mg/L. If DO is low at feeding time, the feed is delayed 30 minutes to prevent waste (uneaten feed pollutes the water and costs 60–70% of production expenses).

4.3.4 Lighting System

Four 20W lights are automatically controlled by the ambient light sensor — turning on when intensity falls below 500 lux (nighttime) and off when bright (daytime). This eliminates the need for manual intervention every day.

CHAPTER 5 — AUTOMATIC CONTROL LOGIC (THRESHOLD-BASED TRIGGER)

Every 5 seconds, Goroutine 2 (Controller) reads the latest sensor values from SQLite and executes threshold-based trigger logic to determine the state of every actuator. The system is reactive — responding to real conditions in real-time, not just following a predetermined schedule.

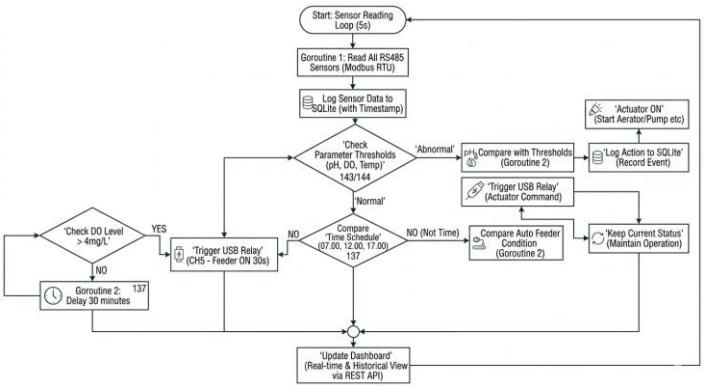


Figure 10. Control Logic Flowchart — Sensor → Decision → Actuator

5.1 Trigger Logic Per Actuator

Aerators #1 & #2 — DO Trigger

- DO < 4.0 mg/L → Both aerators ON simultaneously (critical oxygen level)
- DO > 5.5 mg/L → Both aerators OFF (save SPS energy)
- DO 4.0–5.5 mg/L → Status maintained (hysteresis — prevents rapid on/off cycling)

Inlet Pump — Multi-Parameter Trigger

- pH < 7.5 → Inlet Pump ON (water too acidic)
- pH > 8.5 → Inlet Pump ON (water too alkaline)
- Salinity > 25 ppt → Inlet Pump ON (too saline, dilution needed)
- Water Level < 1.0 m → Inlet Pump ON (critical low level)
- All normal → Inlet Pump OFF

Outlet Pump — Turbidity Trigger

- Turbidity > 50 NTU → Outlet Pump ON (water too murky, bacterial risk)
- Turbidity < 20 NTU → Outlet Pump OFF

Auto Feeder — Schedule + DO Condition

- Schedule 07:00 / 12:00 / 17:00 arrives → system checks DO first
- DO ≥ 4.0 mg/L → Feeder ON for 30 seconds
- DO < 4.0 mg/L → Feeder DELAYED 30 minutes → re-check DO

Lighting — Light Sensor Trigger

- Ambient light < 500 lux → Lights ON (nighttime detected)
- Ambient light ≥ 500 lux → Lights OFF (daytime detected)

5.2 Why Threshold-Based Trigger is Superior to Scheduled Control

Schedule-based control activates actuators only at predetermined times regardless of actual pond conditions. A threshold-based trigger system responds to real conditions at any moment — including a critical DO drop at 2:00 AM while the farmer is sleeping — without any delay. This fundamental difference is precisely what justifies the word 'Smart' in the system's name, and can be scientifically defended during jury presentations.

CHAPTER 6 — DASHBOARD & FARMER ACCESS

The dashboard is a visual interface that allows farmers to monitor all pond conditions and control actuators from anywhere using a smartphone or computer, without physically being present at the pond site.

6.1 Technology Stack & Access Flow

Stack: Go Web Server (port 8080) + HTML/CSS/JavaScript frontend + SQLite + Cloudflare Tunnel → HTTPS custom domain

1. Farmer opens browser → navigates to dashboard.theirdomain.com
2. Enters password (authenticated via 7-day cookie session)
3. Cloudflare forwards the request to Go server on HG680P via encrypted tunnel
4. Go retrieves latest sensor data from SQLite → renders real-time dashboard
5. Farmer monitors conditions or controls actuators manually if needed

6.2 Dashboard Pages

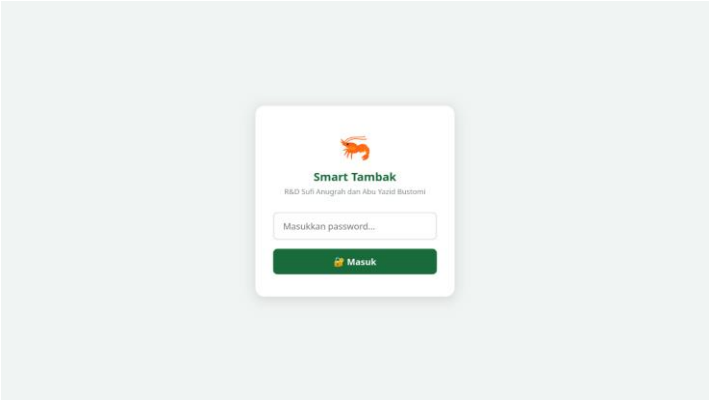


Figure 11. Login Page — Password Authentication Before Dashboard Access

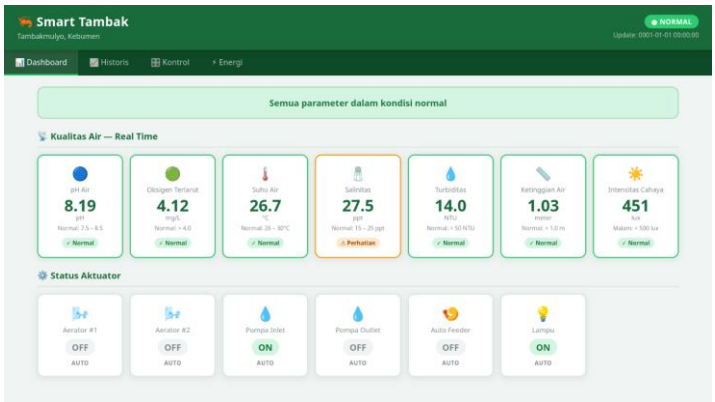
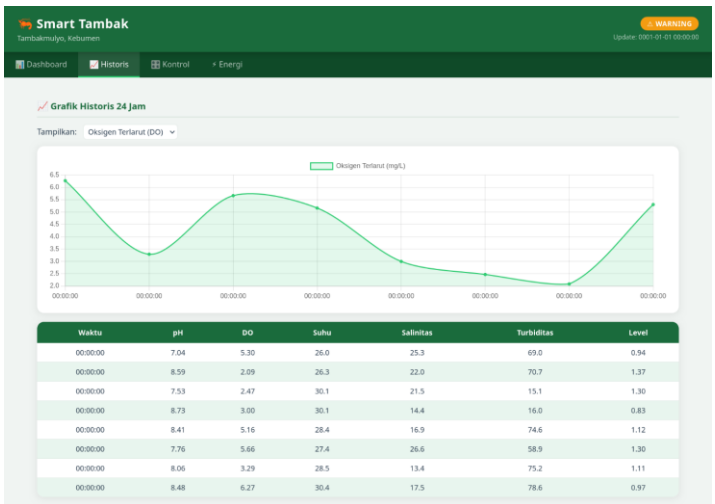


Figure 12. Main Dashboard — Real-Time Sensor Values & Actuator Status



Waktu	pH	DO	Suhu	Salinitas	Turbiditas	Level
00:00:00	7.04	5.30	26.0	25.3	69.0	0.94
00:00:00	8.59	2.09	26.3	22.0	70.7	1.37
00:00:00	7.53	2.47	30.1	21.5	15.1	1.30
00:00:00	8.73	3.00	30.1	14.4	16.0	0.83
00:00:00	8.41	5.16	28.4	16.9	74.6	1.12
00:00:00	7.76	5.66	27.4	26.6	58.9	1.30
00:00:00	8.06	3.29	28.5	13.4	75.2	1.11
00:00:00	8.48	6.27	30.4	17.5	78.6	0.97

Figure 13. Historical Data Page — 24-Hour Sensor Trend Charts (Per-Hour Aggregation)

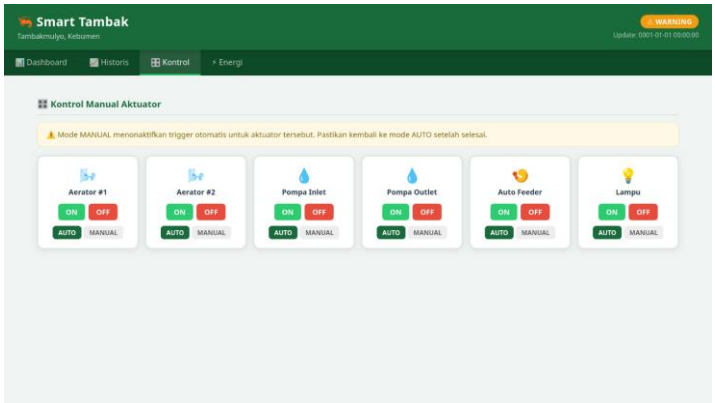


Figure 14. Manual Control Page — AUTO/MANUAL Mode Toggle Per Actuator

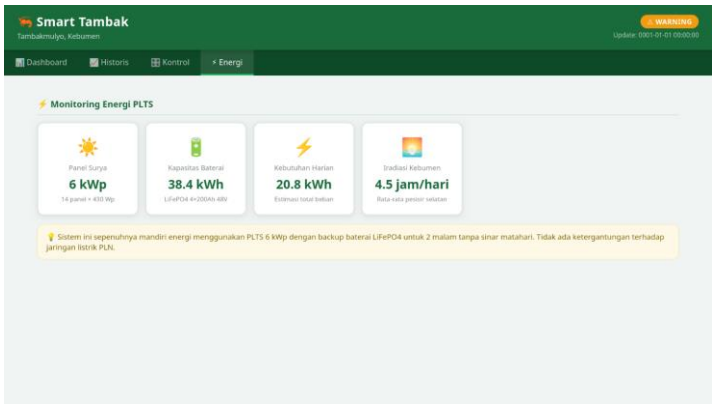


Figure 15. Energy Monitoring Page — SPS Capacity, Daily Consumption & Battery Status

CHAPTER 7 — SOURCE CODE STRUCTURE

The project source code is organized efficiently into 14 files. Complexity lives inside the files, not in the number of files. The full source code is publicly available at:

github.com/sufiarh/sistem-smart-tambak

7.1 Project Tree Structure

```
smart-tambak/
|
|-- main.go                   # Entry point — initialize & launch 3
goroutines
|-- go.mod                    # Go module & dependencies
|-- go.sum                    # Dependency checksum
|-- config.yaml               # Thresholds, schedules, USB ports, sensor
addresses
|-- install.sh                # Automated installation script for Armbian
Linux
|-- smart-tambak.service      # Systemd autostart service on boot
|
|-- core/
|   |-- sensor.go             # Structs, Modbus RTU, RS485 reading goroutine
|   |-- controller.go         # Threshold trigger logic for all actuators
|   |-- relay.go              # USB HID — relay channel control CH1-CH6
|   |-- database.go           # SQLite — connection, migration, save &
retrieve
|   `-- server.go             # Web server, routes, handlers, REST API, auth
|
`-- web/
    |-- index.html            # Single Page App — 4-tab dashboard
    |-- style.css             # Full dashboard styling
    `-- app.js                # Sensor polling, Chart.js graphs, relay
control
```

7.2 Go Dependencies

- github.com/goburrow/modbus — Modbus RTU communication via RS485
- github.com/mattn/go-sqlite3 — SQLite driver for Go
- github.com/karalabe/hid — USB HID Relay Module control
- gopkg.in/yaml.v3 — config.yaml file parsing

CHAPTER 8 — INSTALLATION GUIDE

This guide covers the complete installation process from preparing the STB HG680P to running the system in full production mode. Two modes are available: simulation mode (laptop/PC for development and testing) and production mode (HG680P with real hardware).

8.1 Hardware Requirements

- STB HG680P (or Raspberry Pi / Orange Pi as alternatives)
- MicroSD Card minimum 32 GB Class 10
- USB Relay Module 8 Channel (USB HID protocol)
- USB to RS485 Converter (CH340 or FT232RL chip)
- 7× RS485 Modbus Sensors (pH, DO, Temperature, Salinity, Turbidity, Water Level, Light)
- Laptop/PC with Linux for initial setup and development

8.2 Software & Dependency Requirements

Component	Installation Command / Steps
Armbian Linux	Flash to MicroSD via Balena Etcher, boot on HG680P
Go (Golang) ARM64	wget go1.22.linux-arm64.tar.gz → extract → set PATH
SQLite3	sudo apt install sqlite3 libsqlite3-dev
CH340/RS485 Driver	Already included in Armbian kernel — check: dmesg | grep ttyUSB
Cloudflare Tunnel	wget cloudflared-linux-arm64 → chmod +x → mv to /usr/local/bin
Git	sudo apt install git → clone repository
Go Dependency	go get github.com/goburrow/modbus
	go get github.com/mattn/go-sqlite3
	go get github.com/karalabe/hid
	go get gopkg.in/yaml.v3

8.3 Installation — Simulation Mode (Linux Laptop/PC)

Step 1 — Clone the Repository

```
git clone https://github.com/sufiarh/sistem-smart-tambak
cd sistem-smart-tambak
```

Step 2 — Install Go Dependencies

```
go mod tidy
```

Step 3 — Ensure Simulation Mode is Active in config.yaml

```
app:
  simulation: true   # true = laptop/PC, false = HG680P
production
```

Step 4 — Build & Run

```
go build ./...
go run main.go
```

Step 5 — Access the Dashboard

```
Open browser → http://localhost:8080
Password: petambak
```

8.4 Installation — Production Mode (HG680P)

Step 1 — Flash Armbian to MicroSD

```
# Download Armbian for HG680P from armbian.com
# Flash using Balena Etcher to MicroSD card
# Insert MicroSD into HG680P and power on
```

Step 2 — Install Go for ARM64

```
wget https://go.dev/dl/go1.22.4.linux-arm64.tar.gz
sudo tar -C /usr/local -xzf go1.22.4.linux-arm64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc && go version
```

Step 3 — Install System Dependencies

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install sqlite3 libsqlite3-dev git curl
```

Step 4 — Install Cloudflare Tunnel

```
wget https://github.com/cloudflare/cloudflared/releases/
      latest/download/cloudflared-linux-arm64
chmod +x cloudflared-linux-arm64
sudo mv cloudflared-linux-arm64 /usr/local/bin/cloudflared
cloudflared tunnel login        # Login to your Cloudflare
account
cloudflared tunnel create smart-tambak
```

Step 5 — Clone & Build on HG680P

```
git clone https://github.com/sufiarh/sistem-smart-tambak
cd sistem-smart-tambak
go mod tidy && go build -o smart-tambak main.go
```

Step 6 — Switch to Production Mode in config.yaml

```
app:
  simulation: false   # Activate real hardware

hardware:
  rs485_port: "/dev/ttyUSB0"  # Verify with: dmesg | grep ttyUSB
```

Step 7 — Configure Autostart with Systemd

```
sudo cp smart-tambak.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable smart-tambak
sudo systemctl start smart-tambak
sudo systemctl status smart-tambak   # Check status
```

Step 8 — Verify System is Running

```
# View system logs
journalctl -u smart-tambak -f

# Verify sensor detection
dmesg | grep ttyUSB && ls /dev/ttyUSB*

# Access local dashboard
http://[HG680P-IP-ADDRESS]:8080
```

Pro Tip: To cross-compile from laptop to HG680P without installing Go on the device, use: GOARCH=arm64 GOOS=linux go build -o smart-tambak main.go — the resulting binary can be directly copied to the HG680P and executed.

8.5 Troubleshooting

Sensor not detected

- Run: dmesg | grep ttyUSB — ensure CH340 is detected as /dev/ttyUSB0
- Verify Modbus sensor addresses match config.yaml settings
- Check RS485 wiring: ensure A+ and B- are not swapped

USB Relay not responding

- Run: lsusb — verify Vendor ID 0x16c0 and Product ID 0x05df are listed
- Check config.yaml: relay_vendor_id and relay_product_id must match
- Grant USB access: sudo usermod -a -G dialout $USER

Dashboard not accessible from outside

- Check cloudflared service: systemctl status cloudflared
- Verify internet connectivity on HG680P: ping 8.8.8.8
- List active tunnels: cloudflared tunnel list

CHAPTER 9 — SUMMARY & CONCLUSION

9.1 System Summary

This system repurposes an STB HG680P flashed with Armbian Linux as the main Single Board Computer, housed in a centralized control building alongside a 6 kWp solar power infrastructure comprising 14 solar panels, MPPT SCC, 38.4 kWh LiFePO4 battery bank, and a Pure Sine Wave inverter — providing completely off-grid power independent of PLN. The HG680P runs a single Go binary split into three parallel goroutines — sensor reading, control logic, and web dashboard — with SQLite as the historical data store. Through USB Port 1, the HG680P controls 6 Smart MCBs via an 8-channel USB Relay Module. Through USB Port 2, it reads 7 RS485 sensors in daisy-chain configuration every 5 seconds via Modbus RTU. The system makes autonomous decisions based on threshold-based trigger logic, while the feeder operates on schedule with a DO safety check. The entire system is monitored via a web dashboard accessible from anywhere through a private domain using Cloudflare Tunnel.

9.2 System Advantages

- Energy independence: 6 kWp SPS + 38.4 kWh LiFePO4 — fully off-grid, no PLN dependency
- Hardware innovation: Cheap STB HG680P (USD 12–18) converted into a functional Linux SBC
- No microcontroller: Go on HG680P via USB HID & RS485 — no Arduino or ESP32 involved
- Smart control: Threshold-based trigger responds to actual pond conditions in real-time
- RS485 daisy-chain: 7 sensors on 1 cable via 1 USB port — clean and efficient wiring
- Global access: Cloudflare Tunnel → HTTPS custom domain — secure, no port forwarding
- Sustainability: Solar energy supports environmentally responsible aquaculture practices

- Replicability: Affordable design allows small-scale shrimp farmers across Indonesia to adopt it

9.3 Relevance to Competition Theme

6. Energy Transition: Solar power replaces PLN grid dependency in remote coastal pond areas of Kebumen
7. Energy Efficiency: Automatic trigger-based control ensures aerators and pumps only activate when truly needed
8. Sustainable Aquaculture: Precise water quality monitoring produces healthier, more productive, environmentally responsible shrimp farming

9.4 R&D Information & Project Access

R&D Team	Sufi Anugrah | Abu Yazid Bustomi
Competition	Central Java Youth Sustainability Competition 2026
Category	Scientific Paper (KTI)
Grand Final & Awarding	June 20, 2026
GitHub Repository	github.com/sufiarh/sistem-smart-tambak
Study Case Location	Tambakmulyo, Kebumen, Central Java, Indonesia

9.5 Keywords

Keywords: Internet of Things, Solar Power System, Automatic Control System, Pacific White Shrimp, Kebumen, Energy Transition, Sustainable Aquaculture, RS485 Modbus, Single Board Computer, Go (Golang), STB HG680P, Cloudflare Tunnel